

MI SENSOR API

Version 3.3

©2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020
3.1	- Added MI_SNR_SetAnadecSrcAttr - Deleted MI_SNR_MAX_PADNUM and MI_SNR_MAX_PLANENUM	03/30/2021
3.2	Added PROCFS introduction	08/25/2021
3.3	Update MI_SNR_IntfMode_e Add LVDS	12/22/2021

TABLE OF CONTENTS

REVISION HISTORY i

TABLE OF CONTENTS ii

1. 概述 1

- 1.1. 概述 1
- 1.2. 流程框图 1
- 1.3. 关键字说明 2

2. API 参考 3

- 2.1. MI_SNR_Enable 3
- 2.2. MI_SNR_Disable 5
- 2.3. MI_SNR_GetPadInfo 6
- 2.4. MI_SNR_GetPlaneInfo 7
- 2.5. MI_SNR_GetFps 7
- 2.6. MI_SNR_SetFps 8
- 2.7. MI_SNR_GetAnadecSrcAttr 9
- 2.8. MI_SNR_SetAnadecSrcAttr 10
- 2.9. MI_SNR_QueryResCount 10
- 2.10. MI_SNR_GetRes 11
- 2.11. MI_SNR_GetCurRes 12
- 2.12. MI_SNR_SetRes 13
- 2.13. MI_SNR_SetOrien 14
- 2.14. MI_SNR_GetOrien 15
- 2.15. MI_SNR_SetPlaneMode 16
- 2.16. MI_SNR_GetPlaneMode 17
- 2.17. MI_SNR_CustFunction 17
- 2.18. MI_SNR_InitDev 20
- 2.19. MI_SNR_DeInitDev 20

3. SENSOR 数据类型 22

- 3.1. MI_SNR_PADID 22
- 3.2. MI_SNR_HDRSrc_e 23
- 3.3. MI_SNR_HDRHWMode_e 23
- 3.4. MI_SNR_Res_t 24
- 3.5. MI_SNR_AttrParallel_t 25
- 3.6. MI_SNR_MipiAttr_t 26
- 3.7. MI_SNR_AttrBt656_t 26
- 3.8. MI_SNR_IntfAttr_u 27
- 3.9. MI_SNR_PADInfo_t 28
- 3.10. MI_SNR_PlaneInfo_t 28
- 3.11. MI_SNR_CUST_DIR_e 29
- 3.12. MI_SNR_InitParam_t 30
- 3.13. MI_SNR_Anadec_SrcAttr_t 30
- 3.14. MI_SNR_Anadec_Status_e 31
- 3.15. MI_SNR_Anadec_TransferMode_e 32
- 3.16. MI_SNR_Anadec_Format_e 32

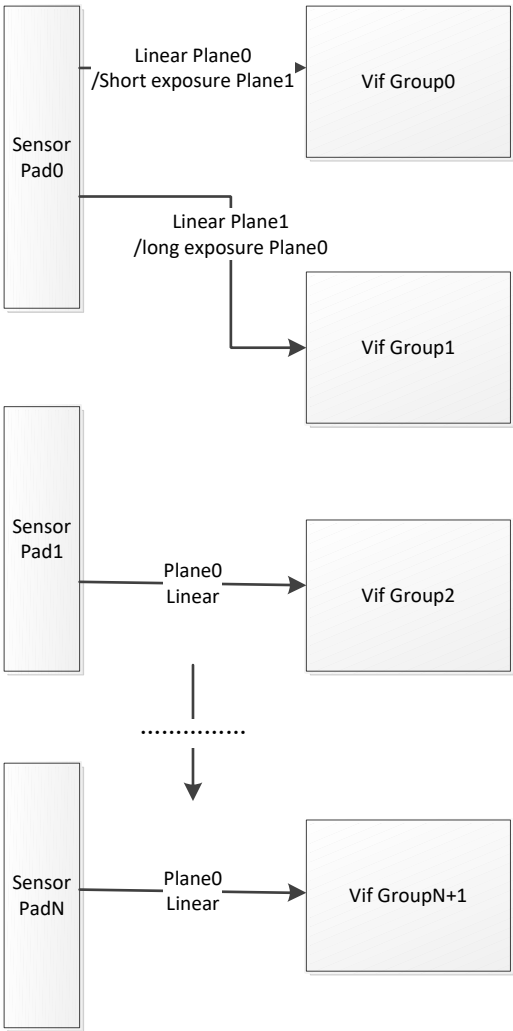
3.17.	MI_SNR_Polar_e	33
3.18.	MI_SNR_SyncAttr_t	33
3.19.	MI_SNR_BitOrder_e	34
3.20.	MI_SNR_ClkEdge_e	35
3.21.	MI_SNR_IntfMode_e	36
3.22.	MI_SNR_HDRTYPE_e	36
4.	SENSOR 错误码	38
5.	PROCFS 介绍	39
5.1.	cat	39
5.2.	echo	41

1. 概述

1.1. 概述

SNR(sensor)模块实现获取摄像头接口信息、调整分辨率和帧率等功能。

1.2. 流程框图



1.3. 关键字说明

- Pad
Sensor 硬件插口位置。
- Plane
Pad 下的通道名称。
- Res
Resolution 分辨率简称。
- Orien
确定方向， 设置 sensor 水平和竖直方向镜像。
- VC
Virtual Channel 虚拟通道。

2. API 参考

API名	功能
MI_SNR_Enable	设置Sensor使能
MI_SNR_Disable	设置Sensor失能
MI_SNR_GetPadInfo	获取Sensor设备信息
MI_SNR_GetPlaneInfo	获取Sensor通道信息
MI_SNR_GetFps	获取Sensor当前帧率
MI_SNR_SetFps	设置Sensor帧率
MI_SNR_GetAnadecSrcAttr	获取模拟信号输入参数
MI_SNR_SetAnadecSrcAttr	设置信号通道的参数
MI_SNR_QueryResCount	获取Sensor支持分辨率的数量
MI_SNR_GetRes	获取索引对应的sensor分辨率
MI_SNR_GetCurRes	获取Sensor当前分辨率
MI_SNR_SetRes	设置Sensor分辨率
MI_SNR_GetOrien	获取Sensor翻转属性
MI_SNR_SetOrien	设置Sensor翻转
MI_SNR_SetPlaneMode	设置Sensor通道模式
MI_SNR_GetPlaneMode	获取设置的Sensor通道模式
MI_SNR_CustFunction	设置Sensor客制化功能
MI_SNR_InitDev	初始化Sensor设备
MI_SNR_DeInitDev	反初始化Sensor设备

2.1. MI_SNR_Enable

➤ 功能

设置 SENSOR 对应设备的使能

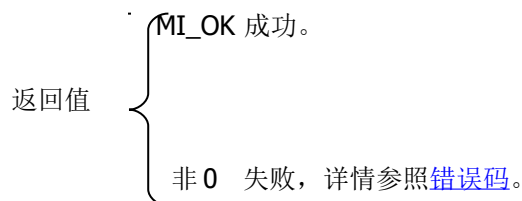
➤ 语法

MI_S32 MI_SNR_Enable([MI_SNR_PADID](#) ePADId);

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR 设备号。 取值范围：[0, MI_SNR_MAX_PAD_NUM]。	输入

➤ 返回值



➤ 依赖

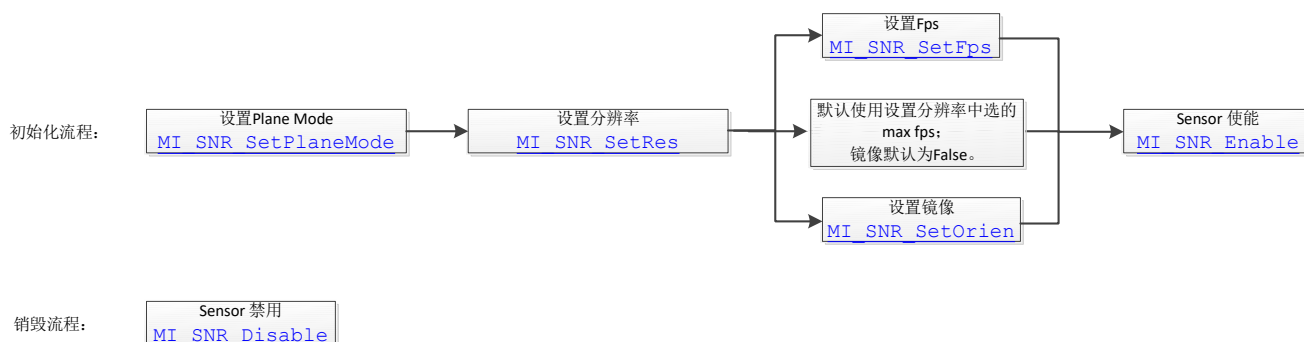
- 头文件：mi_sensor_datatype.h、mi_sensor.h
 - 库文件：libmi_sensor.a

※ 注意

- 在调用前要保证 SENSOR 设备处于未初始化状态。如果 SENSOR 设备已处于使能状态，可以使用 [MI_SNR_Disable](#) 来去初始化设备。
- enable 之前必须要设置 [MI_SNR_SetPlaneMode](#) 和 [MI_SNR_SetRes](#)。
- 由于 MI_SNR 模块没有与 Dram 交互，所以不需要与后端模块进行 sys 间的 bind, 数据流会自动流向 MI_VIF。

➤ 举例

Sensor 初始化运行和退出范例如下：



```

MI_U32 u32ResCount =0;
MI_U8 u8ResIndex =0;
MI_U8 u8ChocieRes =0;
MI_SNR_PADID SnrPad= 0;
MI_SNR_QueryResCount(SnrPad, &u32ResCount);
for(u8ResIndex=0; u8ResIndex < u32ResCount; u8ResIndex++)
{
    MI_SNR_GetRes(SnrPad, u8ResIndex, &stRes);
    printf("index %d, Crop(%d,%d,%d,%d), outputsize(%d,%d), maxfps %d, minfps %d,
    ResDesc %s\n",u8ResIndex, stRes.stCropRect.u16X, stRes.stCropRect.u16Y,
    stRes.stCropRect.u16Width,stRes.stCropRect.u16Height,stRes.stOutputSize.u16Width,
    stRes.stOutputSize.u16Height, stRes.u32MaxFps,stRes.u32MinFps, stRes.strResDesc);
}
  
```

```
printf("select res\n");
scanf("%c", &select);

if(E_MI_SNR_HDR_TYPE_OFF== eHdrType)
{
    MI_SNR_SetPlaneMode(SnrPad, FALSE);
}
else
{
    MI_SNR_SetPlaneMode(SnrPad, TRUE);
}

MI_SNR_SetRes(SnrPad,u8ResIdx);
MI_SNR_Enable(SnrPad);

/*****
/* Exit call interface */
*****/
MI_SNR_Disable(SnrPad);
```

- 相关主题
 - [MI_SNR_Disable](#)

2.2. MI_SNR_Disable

- 功能

设置 SENSOR 对应设备失能。
- 语法

MI_S32 MI_SNR_Disable([MI_SNR_PADID](#) ePADId);
- 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围：[0, MI_SNR_MAX_PAD_NUM)。	输入

➤ 返回值

返回值 {
MI_OK 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_sensor_datatype.h、mi_sensor.h
- 库文件：libmi_sensor.a

➤ 举例

参考 [MI_SNR_Enable](#) 举例。

➤ 相关主题

[MI_SNR_Enable](#)

2.3. MI_SNR_GetPadInfo

➤ 功能

获取 SENSOR 设备信息。

➤ 语法

MI_S32 MI_SNR_GetPadInfo([MI_SNR_PADID](#) ePADId, [MI_SNR_PADInfo_t](#) *pstPadInfo);

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围：[0, MI_SNR_MAX_PAD_NUM)。	输入
pstPadInfo	SENSOR设备属性指针	输出

➤ 返回值

返回值 {
MI_OK 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_sensor_datatype.h、mi_sensor.h
- 库文件：libmi_sensor.a

➤ 相关主题

无。

2.4. MI_SNR_GetPlaneInfo

➤ 功能

获取 SENSOR 通道信息。

➤ 语法

```
MI_S32 MI_SNR_GetPlaneInfo(MI\_SNR\_PADID ePADId, MI_U32 u32PlaneID, MI_SNR_PlaneInfo_t *pstPlaneInfo);
```

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围: [0, MI_SNR_MAX_PAD_NUM)。	输入
u32PlaneID	SENSOR通道号。 取值范围: [0, MI_SNR_MAX_PLANE_NUM)。	输入
pstPlaneInfo	SENSOR通道信息。	输出

➤ 返回值

返回值	MI_OK 成功。
	非 0 失败, 详情参照 错误码 。

➤ 依赖

- 头文件: mi_sensor_datatype.h、mi_sensor.h
- 库文件: libmi_sensor.a

➤ 相关主题

无。

2.5. MI_SNR_GetFps

➤ 功能

获取 SENSOR 帧率

➤ 语法

```
MI_S32 MI_SNR_GetFps(MI\_SNR\_PADID ePADId, MI_U32 *pFps);
```

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围: [0, MI_SNR_MAX_PAD_NUM)。	输入
pFps	帧率指针	输出

➤ 返回值

返回值 {
MI_OK 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_sensor_datatype.h、mi_sensor.h
- 库文件：libmi_sensor.a

※ 注意

获取到的 fps 范围为：
 $\text{min} \times 1000 < \text{fps} < \text{max} \times 1000$ ：精确到小数点后 3 位。

➤ 相关主题

[MI_SNR_SetFps](#)

2.6. MI_SNR_SetFps

➤ 功能

设置 SENSOR 帧率。

➤ 语法

MI_S32 MI_SNR_SetFps([MI_SNR_PADID](#) ePADId, MI_U32 u32Fps);

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围：[0, MI_SNR_MAX_PAD_NUM)。	输入
u32Fps	帧率	输入

➤ 返回值

返回值 {
MI_OK 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_sensor_datatype.h、mi_sensor.h
- 库文件：libmi_sensor.a

※ 注意

- fps 有两个取值区间：

- $\text{min} < \text{fps} < \text{max}$: 精确到个位数
- $\text{min} * 1000 < \text{fps} < \text{max} * 1000$: 精确到小数点后 3 位。
- Fps 的最大/最小值为 [MI_SNR_SetRes](#) 时设置分辨率 Index 所对应的 max/min fps。

➤ 相关主题

[MI_SNR_GetFps](#)

2.7. MI_SNR_GetAnadecSrcAttr

➤ 功能

获取模拟信号输入参数。

➤ 语法

```
MI_S32 MI_SNR_GetAnadecSrcAttr(MI\_SNR\_PADID ePADId,  
MI_U32 u32PlaneID, MI\_SNR\_Anadec\_SrcAttr\_t *pstSrcAttr);
```

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围: [0, MI_SNR_MAX_PAD_NUM)。	输入
u32PlaneID	SENSOR通道号。 取值范围: [0, MI_SNR_MAX_PLANE_NUM)。	输入
pstSrcAttr	源输入参数	输出

➤ 返回值

返回值

- MI_OK 成功。
- 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件: [mi_sensor_datatype.h](#)、[mi_sensor.h](#)
- 库文件: [libmi_sensor.a](#)

※ 注意

在输出为模拟信号的 sensor 上才可以用。

➤ 相关主题

[MI_SNR_Anadec_SrcAttr_t](#)

2.8. MI_SNR_SetAnadecSrcAttr

➤ 功能

设置信号通道的参数。

➤ 语法

```
MI_S32 MI_SNR_SetAnadecSrcAttr(MI\_SNR\_PADID ePADId,  
MI_U32 u32PlaneID, MI\_SNR\_Anadec\_SrcAttr\_t *pstSrcAttr);
```

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围: [0, MI_SNR_MAX_PAD_NUM)。	输入
u32PlaneID	SENSOR通道号。 取值范围: [0, MI_SNR_MAX_PLANE_NUM)。	输入
pstSrcAttr	信号源参数	输入

➤ 返回值

返回值

- MI_OK 成功。
- 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件: [mi_sensor_datatype.h](#)、[mi_sensor.h](#)
- 库文件: [libmi_sensor.a](#)

※ 注意

该接口适用于一个 sensor pad 上有多组信号设置的情况;
例如一个 BT656 上有 4 个 AHD sensor， 设置每个 sensor 的分辨率。

➤ 相关主题

[MI_SNR_Anadec_SrcAttr_t](#)

2.9. MI_SNR_QueryResCount

➤ 功能

获取 SENSOR 支持分辨率的数量。

➤ 语法

```
MI_S32 MI_SNR_QueryResCount(MI\_SNR\_PADID ePADId,  
MI_U32 *pu32ResCount);
```

➤ 形参

参数名称	描述	输入/输出
ePADId	SENSOR设备号。 取值范围：[0, MI_SNR_MAX_PAD_NUM)。	输入
*pu32ResCount	SENSOR设备支持的resolution 数量	输出

- 返回值
- 返回值

MI_OK 成功。

非 0 失败，详情参照[错误码](#)。

- 依赖
- 头文件：mi_sensor_datatype.h、mi_sensor.h
 - 库文件：libmi_sensor.a

※ 注意

无。

- 举例
- 参考 [MI_SNR_GetRes](#) 举例

- 相关主题
- [MI_SNR_GetRes](#)

2.10. MI_SNR_GetRes

- 功能
- 获取 resolution 映射表中索引对应的分辨率。

- 语法
- ```
MI_S32 MI_SNR_GetRes(MI_SNR_PADID ePADId, MI_U8 u8ResIdx, MI_SNR_Res_t *pstRes);
```

- 形参

| 参数名称     | 描述                                                           | 输入/输出 |
|----------|--------------------------------------------------------------|-------|
| ePADId   | SENSOR设备号。<br>取值范围：[0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| u8ResIdx | 分辨率映射表中的索引                                                   | 输入    |
| *pstRes  | 序号所对应的分辨率                                                    | 输出    |

- 返回值
- 返回值

MI\_OK 成功。



非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi\_sensor\_datatype.h、mi\_sensor.h
- 库文件：libmi\_sensor.a

※ 注意

无

➤ 举例

获取 resolution 列表，以及选择对应分辨率设置范例如下：

```
MI_U32 u32ResCount =0;
MI_U8 u8ResIndex =0;
MI_U8 u8ChocieRes =0;
MI_SNR_PADID SnrPad= 0;
MI_SNR_QueryResCount(SnrPad, &u32ResCount);
for(u8ResIndex=0; u8ResIndex < u32ResCount; u8ResIndex++)
{
 MI_SNR_GetRes(SnrPad, u8ResIndex, &stRes);
 printf("index %d, Crop(%d,%d,%d,%d), outputsize(%d,%d), maxfps %d, minfps %d,
 ResDesc %s\n",u8ResIndex, stRes.stCropRect.u16X, stRes.stCropRect.u16Y,
 stRes.stCropRect.u16Width,stRes.stCropRect.u16Height,stRes.stOutputSize.u16Width,
 stRes.stOutputSize.u16Height,stRes.u32MaxFps,stRes.u32MinFps,stRes.strResDesc);
}

printf("select res\n");
scanf("%c", &select);
MI_SNR_SetRes(SnrPad,u8ResIdx);
MI_SNR_GetCurRes(SnrPad, &u8ResIndex, &stRes);
```

➤ 相关主题

[MI\\_SNR\\_QueryResCount](#)

[MI\\_SNR\\_Res\\_t](#)

## 2.11. MI\_SNR\_GetCurRes

➤ 功能

获取 sensor 当前分辨率和在分辨率映射表中的位置。

➤ 语法

```
MI_S32 MI_SNR_GetCurRes(MI_SNR_PADID ePADId, MI_U8 *pu8CurResIdx,MI_SNR_Res_t
*pstCurRes);
```

## ➤ 形参

| 参数名称          | 描述                                                           | 输入/输出 |
|---------------|--------------------------------------------------------------|-------|
| ePADId        | SENSOR设备号。<br>取值范围：[0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| *pu8CurResIdx | 当前分辨率的索引                                                     | 输出    |
| *pstCurRes    | 当前分辨率信息                                                      | 输出    |

## ➤ 返回值

|     |                                   |
|-----|-----------------------------------|
| 返回值 | MI_OK 成功。                         |
|     | 非 0 失败，详情参照 <a href="#">错误码</a> 。 |

## ➤ 依赖

- 头文件：mi\_sensor\_datatype.h、mi\_sensor.h
- 库文件：libmi\_sensor.a

## ※ 注意

无。

## ➤ 举例

参考 [MI\\_SNR\\_GetRes](#) 举例

## ➤ 相关主题

[MI\\_SNR\\_Res\\_t](#)

## 2.12. MI\_SNR\_SetRes

## ➤ 功能

设置 sensor 设备输出分辨率

## ➤ 语法

```
MI_S32 MI_SNR_SetRes(MI_SNR_PADID ePADId, MI_U8 u8ResIdx);
```

## ➤ 形参

| 参数名称     | 描述                                                           | 输入/输出 |
|----------|--------------------------------------------------------------|-------|
| ePADId   | SENSOR设备号。<br>取值范围：[0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| u8ResIdx | Dev Num                                                      | 输入    |

## ➤ 返回值

|     |           |
|-----|-----------|
| 返回值 | MI_OK 成功。 |
|     |           |

非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi\_sensor\_datatype.h、mi\_sensor.h
- 库文件：libmi\_sensor.a

※ 注意

在 [MI\\_SNR\\_Enable](#) 之后不能单独 call 此 API，要用切换 Sensor resolution 的流程切换 Sensor 分辨率。

➤ 举例

参考 [MI\\_SNR\\_GetRes](#) 举例

➤ 相关主题

[MI\\_SNR\\_GetRes](#)

[MI\\_SNR\\_Res\\_t](#)

## 2.13. MI\_SNR\_SetOrien

➤ 功能

设置 sensor 图象翻转属性

➤ 语法

MI\_S32 MI\_SNR\_SetOrien([MI\\_SNR\\_PADID](#) ePADId, MI\_BOOL bMirror, MI\_BOOL bFlip);

➤ 形参

| 参数名称    | 描述                                                           | 输入/输出 |
|---------|--------------------------------------------------------------|-------|
| ePADId  | SENSOR设备号。<br>取值范围：[0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| bMirror | 使能竖直镜像翻转                                                     | 输入    |
| bFlip   | 使能水平镜像翻转                                                     | 输入    |

➤ 返回值

返回值

- MI\_OK 成功。
- 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi\_sensor.h
- 库文件：libmi\_sensor.a

※ 注意

无。

➤ 举例

该 API 可以单独使用。

➤ 相关主题

[MI\\_SNR\\_GetOrien](#)

## 2.14. MI\_SNR\_GetOrien

➤ 功能

获取 sensor 图象翻转属性

➤ 语法

```
MI_S32 MI_SNR_GetOrien(MI_SNR_PADID ePADId, MI_BOOL *pbMirror, MI_BOOL *pbFlip);
```

➤ 形参

| 参数名称      | 描述                                                            | 输入/输出 |
|-----------|---------------------------------------------------------------|-------|
| ePADId    | SENSOR设备号。<br>取值范围: [0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| *pbMirror | 使能竖直镜像翻转                                                      | 输出    |
| *pbFlip   | 使能水平镜像翻转                                                      | 输出    |

➤ 返回值

返回值

- MI\_OK 成功。
- 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件: `mi_sensor.h`
- 库文件: `libmi_sensor.a`

※ 注意

无。

➤ 举例

无。

➤ 相关主题

[MI\\_SNR\\_SetOrien](#)

## 2.15. MI\_SNR\_SetPlaneMode

➤ 功能

设置 sensor Plane 模式。

➤ 语法

MI\_S32 MI\_SNR\_SetPlaneMode([MI\\_SNR\\_PADID](#) ePADId, MI\_BOOL bEnable);

➤ 形参

| 参数名称    | 描述                                                            | 输入/输出 |
|---------|---------------------------------------------------------------|-------|
| ePADId  | SENSOR设备号。<br>取值范围: [0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| bEnable | 开HDR 需要置为TRUE, 否则为FALSE                                       | 输入    |

➤ 返回值

返回值

- MI\_OK 成功。
- 非 0 失败, 详情参照[错误码](#)。

➤ 依赖

- 头文件: `mi_sensor.h`
- 库文件: `libmi_sensor.a`

※ 注意

plane 和 SensorPad 之间有两种关系, 当 PlaneMode 为 False 时, SensorPad 和 Plane 之间是一一对应的关系;当 PlaneMode 为 True 时, SensorPad 一个对应多个 Plane。由于 Hdr Mode 时需要用到两个 Plane 分别接收长曝和短曝, 所以 Plane mode 应该设置为 True。

➤ 举例

```
MI_SNR_PADID SnrPad= 0;
if(E_MI_VIF_HDR_TYPE_OFF== eHdrType)
{
 MI_SNR_SetPlaneMode(SnrPad, FALSE);
}
else
{
 MI_SNR_SetPlaneMode(SnrPad, TRUE);
}
```

➤ 相关主题

[MI\\_SNR\\_GetPlaneMode](#)

## 2.16. MI\_SNR\_GetPlaneMode

➤ 功能

获取上层设置的 Sensor Plane 模式。

➤ 语法

MI\_S32 MI\_SNR\_GetPlaneMode([MI\\_SNR\\_PADID](#) ePADId, MI\_BOOL \*pbEnable);

➤ 形参

| 参数名称      | 描述                                                            | 输入/输出 |
|-----------|---------------------------------------------------------------|-------|
| ePADId    | SENSOR设备号。<br>取值范围: [0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| *pbEnable | 开HDR 需要置为TRUE, 否则为FALSE                                       | 输出    |

➤ 返回值

|     |                                    |
|-----|------------------------------------|
| 返回值 | MI_OK 成功。                          |
|     | 非 0 失败, 详情参照 <a href="#">错误码</a> 。 |

➤ 依赖

- 头文件: `mi_sensor.h`
- 库文件: `libmi_sensor.a`

※ 注意

无。

➤ 举例

无。

➤ 相关主题

[MI\\_SNR\\_SetPlaneMode](#)

## 2.17. MI\_SNR\_CustFunction

➤ 功能

设置 Sensor 客制化功能。例如需要对 sensor 寄存器需要进行读写, 或者某些特殊 sensor 有传感器功能通过该 api 获取数据等。

➤ 语法

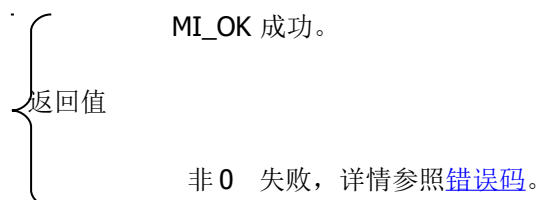
MI\_S32 MI\_SNR\_CustFunction([MI\\_SNR\\_PADID](#) ePADId, MI\_U32 u32CmdId, MI\_U32 u32DataSize, void \*pCustData, [MI\\_SNR\\_CUST\\_DIR\\_e](#) eDir);

➤ 形参

| 参数名称     | 描述                                                            | 输入/输出 |
|----------|---------------------------------------------------------------|-------|
| ePADId   | SENSOR设备号。<br>取值范围: [0, <a href="#">MI_SNR_MAX_PAD_NUM</a> )。 | 输入    |
| u32CmdId | 客制化功能ID                                                       | 输入    |

| 参数名称        | 描述                  | 输入/输出 |
|-------------|---------------------|-------|
| u32DataSize | 客制化功能数据 buffer size | 输入    |
| pCustData   | 客制化功能数据 buffer      | 输入    |
| eDir        | 客制化数据类型             | 输入    |

➤ 返回值



➤ 依赖

- 头文件：mi\_sensor.h
- 库文件：libmi\_sensor.a

※ 注意

和 sensor driver 中的 pCus\_sensor\_CustDefineFunction api 接口对应

➤ 举例

对 sensor register 读写范例如下：

APP 中实现如下：

```
#define I2C_READ (0x01)
```

```
#define I2C_WRITE (0x02)
```

```
typedef struct stI2CRegData_s
```

```
{
```

```
 MI_U16 u16Reg;
```

```
 MI_U16 u16Data;
```

```
}stI2CRegData_t;
```

```
MI_SNR_PADID SnrPad= 0;
```

```
stI2CRegData_t stReadReg;
```

```
stI2CRegData_t stWriteReg;
```

```
MI_U16 u16DataSize=sizeof(stI2CRegData_t);
```

```
memset(&stReadReg, 0x0, sizeof(stI2CRegData_t));
```

```
memset(&stWriteReg, 0x0, sizeof(stI2CRegData_t));
```

```
stReadReg.u16Reg = 0x3007;
```

```
MI_SNR_CustFunction(SnrPad, I2C_READ, u16DataSize, &stReadReg, E_MI_SNR_CUSTDATA_TO_USER);
```

```
stWriteReg.u16Reg = 0x3007;
```

```
stWriteReg.u16Data = 0x03;
```

```
MI_SNR_CustFunction(SnrPad, I2C_WRITE, u16DataSize, &stWriteReg, E_MI_SNR_CUSTDATA_TO_DRIVER);
```

sensor driver 中实现如下函数：

```
#define I2C_READ (0x01)
#define I2C_WRITE (0x02)

typedef struct stI2CRegData_s
{
 MI_U16 u16Reg;
 MI_U16 u16Data;
}stI2CRegData_t;

static int pCus_sensor_CustDefineFunction(ms_cus_sensor *handle, u32 cmd_id, void *param)
{
 switch(cmd_id)
 {
 case I2C_READ:
 {
 stI2CRegData_t *pRegData = (stI2CRegData_t *)param;
 SensorReg_Read(pRegData->u16Reg, pRegData->u16Data);
 }
 break;
 case I2C_WRITE:
 {
 stI2CRegData_t *pRegData = (stI2CRegData_t *)param;
 SensorReg_Write(pRegData->u16Reg, pRegData->u16Data);
 }
 break;
 default:
 printk("cmdid %d, unknow \n");
 break
 }

 return SUCCESS;
}
```

➤ 相关主题

[MI\\_SNR\\_CUST\\_DIR\\_e](#)

## 2.18. MI\_SNR\_InitDev

➤ 描述

初始化 Sensor 设备。

➤ 语法



```
MI_S32 MI_SNR_InitDev(MI_SNR_InitParam_t *pstInitParam);
```

➤ 参数

| 参数名称         | 描述       | 输入/输出 |
|--------------|----------|-------|
| pstInitParam | 设备初始化参数。 | 输入    |

➤ 返回值

返回值 { MI\_OK 成功。  
非 MI\_OK 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi\_common.h、mi\_sensor.h
- 库文件：libmi\_sensor.a

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

## 2.19. MI\_SNR\_DeInitDev

➤ 描述

反初始化 sensor 设备。

➤ 语法

```
MI_S32 MI_SNR_DeInitDev(void);
```

➤ 参数

无。

➤ 返回值

返回值 { MI\_OK 成功。  
非 MI\_OK 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi\_common.h、mi\_sensor.h
- 库文件：libmi\_sensor.a

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

### 3. SENSOR 数据类型

视频输入相关数据类型定义如下：

| 数据类型                                         | 定义                    |
|----------------------------------------------|-----------------------|
| <a href="#">MI_SNR_PADID</a>                 | 定义Sensor Pad Id类型     |
| <a href="#">MI_SNR_HDRSrc_e</a>              | 定义Sensor HDR 通道序号     |
| <a href="#">MI_SNR_HDRHWMode_e</a>           | 定义 HDR 硬件设置模式         |
| <a href="#">MI_SNR_Res_t</a>                 | 定义 Sensor分辨率属性        |
| <a href="#">MI_SNR_AttrParallel_t</a>        | 定义 Parallel Sensor属性  |
| <a href="#">MI_SNR_MipiAttr_t</a>            | 定义Mipi Sensor属性       |
| <a href="#">MI_SNR_AttrBt656_t</a>           | 定义Bt656 Sensor属性      |
| <a href="#">MI_SNR_IntfAttr_u</a>            | 定义Sensor接口联合体         |
| <a href="#">MI_SNR_PADInfo_t</a>             | 定义Sensor Pad信息        |
| <a href="#">MI_SNR_PlaneInfo_t</a>           | 定义Sensor 通道信息         |
| <a href="#">MI_SNR_CUST_DIR_e</a>            | 定义Sensor 客制化功能数据类型    |
| <a href="#">MI_SNR_InitParam_t</a>           | 定义Sensor 设备初始化参数      |
| <a href="#">MI_SNR_Anadec_SrcAttr_t</a>      | 定义模拟信号输入参数            |
| <a href="#">MI_SNR_Anadec_Status_e</a>       | 定义模拟信号状态参数            |
| <a href="#">MI_SNR_Anadec_TransferMode_e</a> | 定义模拟信号传输模式            |
| <a href="#">MI_SNR_Anadec_Format_e</a>       | 定义模拟信号传输制式            |
| <a href="#">MI_SNR_Polar_e</a>               | 定义信号极性                |
| <a href="#">MI_SNR_SyncAttr_t</a>            | 定义同步信号属性              |
| <a href="#">MI_SNR_BitOrder_e</a>            | 定义视频设备的 data bit 翻转顺序 |
| <a href="#">MI_SNR_ClkEdge_e</a>             | 定义视频设备接收的时钟类型         |
| <a href="#">MI_SNR_IntfMode_e</a>            | 定义视频输入设备的接口模式         |
| <a href="#">MI_SNR_HDRType_e</a>             | 定义视频设备HDR 类型          |

#### 3.1. MI\_SNR\_PADID

➤ 说明

Sensor Pad Id 类型。

## ➤ 定义

```
typedef MI_U32 MI_SNR_PADID;
```

## ※ 注意事项

和硬件上 Sensor Pad 接口相对应。

## ➤ 相关数据类型及接口

无。

### 3.2. MI\_SNR\_HDRSrc\_e

## ➤ 说明

在 HdrMode 时，Plane 通道对应的 Virtual channel。

## ➤ 定义

```
typedef enum
{
 E_MI_SNR_HDR_SOURCE_VC0,
 E_MI_SNR_HDR_SOURCE_VC1,
 E_MI_SNR_HDR_SOURCE_VC2,
 E_MI_SNR_HDR_SOURCE_VC3,
 E_MI_SNR_HDR_SOURCE_MAX
} MI_SNR_HDRSrc_e;
```

## ※ 注意事项

无。

## ➤ 相关数据类型及接口

[MI\\_SNR\\_PlaneInfo\\_t](#)

### 3.3. MI\_SNR\_HDRHWMode\_e

## ➤ 说明

定义 Sensor 硬件设置 HDR 模式。

## ➤ 定义

```
typedef enum
{
 E_MI_SNR_HDR_HW_MODE_NONE = 0,
 E_MI_SNR_HDR_HW_MODE_SONY_DOL = 1,
 E_MI_SNR_HDR_HW_MODE_DCG = 2,
 E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW8 = 3,
 E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW10 = 4,
 E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW12 = 5,
 E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW14 = 6,
```

```
E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW16 = 7,
} MI_SNR_HDRHWMode_e;
```

➤ 成员

| 成员名称                                | 描述                                 |
|-------------------------------------|------------------------------------|
| E_MI_SNR_HDR_HW_MODE_NONE           | 没有开HDR模式                           |
| E_MI_SNR_HDR_HW_MODE_SONY_DOL       | Digital Overlap High Dynamic Range |
| E_MI_SNR_HDR_HW_MODE_DCG            | 双转换增益模式                            |
| E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW8  | 8bit 压缩模式                          |
| E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW10 | 10bit 压缩模式                         |
| E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW12 | 12bit 压缩模式                         |
| E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW14 | 14bit 压缩模式                         |
| E_MI_SNR_HDR_HW_MODE_EMBEDDED_RAW16 | 16bit 压缩模式                         |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_MipiAttr\\_t](#)

### 3.4. MI\_SNR\_Res\_t

➤ 说明

定义 Sensor 分辨率属性

➤ 定义

```
typedef struct MI_SNR_Res_s
{
 MI_SYS_WindowRect_t stCropRect;
 MI_SYS_WindowSize_t stOutputSize; /**< Sensor actual output size */

 MI_U32 u32MaxFps; /**< Max fps in this resolution */
 MI_U32 u32MinFps; /**< Min fps in this resolution*/
 MI_S8 strResDesc[32]; // Need to put "HDR" here if the resolution is for HDR
} __attribute__((packed, aligned(4))) MI_SNR_Res_t;
```

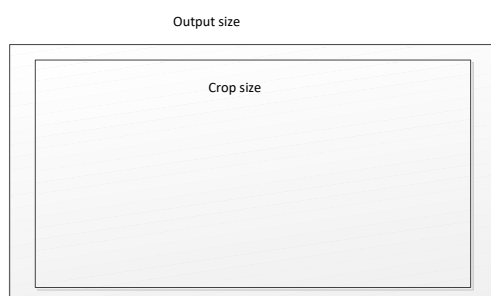
➤ 成员

| 成员名称       | 描述                 |
|------------|--------------------|
| stCropRect | 在output size上裁剪的区域 |

| 成员名称         | 描述                |
|--------------|-------------------|
| stOutputSize | Sensor 输出大小范围     |
| u32MaxFps    | 当前分辨率下最大帧率        |
| u32MinFps    | 当前分辨率下最小帧率        |
| strResDesc   | Resolution string |

※ 注意事项

stOutputSize 为 Sensor 的原始宽高，stCropRect 为在原始图像上裁剪的大小，所以 stCropRect 为 Sensor 的实际输出区域。



➤ 相关数据类型及接口

[MI\\_SNR\\_GetRes](#)

[MI\\_SNR\\_GetCurRes](#)

[MI\\_SNR\\_SetRes](#)

### 3.5. MI\_SNR\_AttrParallel\_t

➤ 说明

定义 parallel sensor 属性。

➤ 定义

```
typedef struct MI_SNR_AttrParallel_s
{
 MI_SNR_SyncAttr_t stSyncAttr;
} MI_SNR_AttrParallel_t;
```

➤ 成员

| 成员名称       | 描述     |
|------------|--------|
| stSyncAttr | 同步信号属性 |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_IntfAttr\\_u](#)

### 3.6. MI\_SNR\_MipiAttr\_t

➤ 说明

定义 Mipi sensor 属性。

➤ 定义

```
typedef struct MI_SNR_MipiAttr_s
{
 MI_U32 u32LaneNum; // 几条信号同时传
 MI_U32 u32DataFormat; //0: YUV 422 format. 1: RGB pattern.
 MI_U32 u32HsyncMode;
 MI_U32 u32Sampling_delay;
 /** < MIPI start sampling delay */ /*bit 0~7: clk_skip_ns. bit 8~15: data_skip_ns*/
 MI_SNR_HDRHWMode_e eHdrHWmode;
 MI_U32 u32Hdr_Virchn_num;
 MI_U32 u32Long_packet_type[2];
}MI_SNR_MipiAttr_t;
```

➤ 成员

| 成员名称                   | 描述                                |
|------------------------|-----------------------------------|
| u32LaneNum             | 支持同时传输数据的信号线数量                    |
| u32DataFormat          | 0: YUV 422 format. 1: RGB pattern |
| u32HsyncMode           | 0:同步前一条<br>1:同步后一条line的hsync 信号   |
| u32Sampling_delay      | 延时跳过数据头部分                         |
| eHdrHWmode             | Sensor支持的HDR mode                 |
| u32Hdr_Virchn_num      | Sensor支持的HDR 虚拟通道数量               |
| u32Long_packet_type[2] | Sensor支持的数据打包格式                   |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_IntfAttr\\_u](#)

### 3.7. MI\_SNR\_AttrBt656\_t

➤ 说明

定义 BT656 sensor 属性。

➤ 定义

```
typedef struct MI_SNR_AttrBt656_s
{
 MI_U32 u32Multiplex_num;
 MI_SNR_SyncAttr_t stSyncAttr;
```

```

 MI_SNR_ClkEdge_e eClkEdge;
 MI_SNR_BitOrder_e eBitSwap;
} MI_SNR_AttrBt656_t;

```

➤ 成员

| 成员名称             | 描述      |
|------------------|---------|
| u32Multiplex_num | 复合模式的路数 |
| stSyncAttr       | 同步信号属性  |
| eClkEdge         | 采样时钟模式  |
| eBitSwap         | 数据排列方向  |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_IntfAttr\\_u](#)

### 3.8. MI\_SNR\_IntfAttr\_u

➤ 说明

定义 sensor 接口类型联合体。

➤ 定义

```

typedef union {
 MI_SNR_AttrParallel_t stParallelAttr;
 MI_SNR_MipiAttr_t stMipiAttr;
 MI_SNR_AttrBt656_t stBt656Attr;
} MI_SNR_IntfAttr_u;

```

➤ 成员

| 成员名称           | 描述                 |
|----------------|--------------------|
| stParallelAttr | Parallel sensor 属性 |
| stMipiAttr     | Mipi sensor属性      |
| stBt656Attr    | Bt656 sensor属性     |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_PADInfo\\_t](#)

### 3.9. MI\_SNR\_PADInfo\_t

➤ 说明



定义 sensor Pad 信息属性。

➤ 定义

```
typedef struct MI_SNR_PADInfo_s
{
 MI_U32 u32PlaneCount;
 MI_SNR_IntfMode_e eIntfMode;
 MI_SNR_HDRType_e eHDRMode;
 MI_SNR_IntfAttr_u unIntfAttr;
 MI_BOOL bEarlyInit;
} MI_SNR_PADInfo_t;
```

➤ 成员

| 成员名称          | 描述                                           |
|---------------|----------------------------------------------|
| u32PlaneCount | BT656 sensor 代表最大复合路数<br>Mipi sensor 代表长短曝数量 |
| eIntfMode     | Sensor 接口枚举                                  |
| eHDRMode      | Hdr 模式                                       |
| unIntfAttr    | Sensor 接口属性联合体                               |
| bEarlyInit    | Sensor 是否已经提前初始化                             |

※ 注意事项

在 dualos 系统中 bEarlyInit 为 True， Pure Linux 下位 False。

➤ 相关数据类型及接口

[MI\\_SNR\\_GetPadInfo](#)

### 3.10. MI\_SNR\_PlaneInfo\_t

➤ 说明

定义 sensor 通道 信息属性。

➤ 定义

```
typedef struct MI_SNR_PlaneInfo_s
{
 MI_U32 u32PlaneID;// For HDR long/short exposure or BT656 channel 0~3
 MI_S8 s8SensorName[32];
 MI_SYS_WindowRect_t stCapRect;
 MI_SYS_BayerId_e eBayerId;
 MI_SYS_DataPrecision_e ePixPrecision;
 MI_SNR_HDRSrc_e eHdrSrc;
 MI_U32 u32ShutterUs;
 MI_U32 u32SensorGainX1024;
 MI_U32 u32CompGain;
```

```
MI_SYS_PixelFormat_e ePixel;
} MI_SNR_PlaneInfo_t;
```

### ➤ 成员

| 成员名称               | 描述                     |
|--------------------|------------------------|
| u32PlaneID         | 通道代号                   |
| s8SensorName       | Sensor name字符串         |
| stCapRect          | 在sensor 数据上裁剪的位置       |
| eBayerId           | RGB 排列顺序               |
| ePixPrecision      | RGB 压缩模式               |
| eHdrSrc            | HDR 通道号                |
| u32ShutterUs       | Sensor Shutter         |
| u32SensorGainX1024 | Sensor Gain            |
| u32CompGain        | Sensor Compensate Gain |
| ePixel             | Sensor 数据格式类型          |

### ※ 注意事项

- 当 mipi 接口 不开 Hdr 时 u32PlaneID = 0xff;开 HDR 时 u32PlaneID =0 代表长曝光，u32PlaneID =1 代表短曝光。
- 当 BT656 接口时，代表当前 Plane 在复合路数中的通道 id。
- #define RGB\_BAYER\_PIXEL(BitMode, PixelID) (E\_MI\_SYS\_PIXEL\_FRAME\_RGB\_BAYER\_BASE+ BitMode\*E\_MI\_SYS\_PIXEL\_BAYERID\_MAX+ PixelID)。
- 通过 sys 接口将 sensor 的 eBayerId 和 ePixPrecision 转换成 sys 的 pixel format，设置给后端 MI\_VIF output 和 MI\_ISP input。
- MI\_SYS\_PixelFormat\_e ePixel = RGB\_BAYER\_PIXEL(ePixPrecision, eBayerId);

### ➤ 相关数据类型及接口

MI\_SNR\_GetPadInfo

## 3.11. MI\_SNR\_CUST\_DIR\_e

### ➤ 说明

定义 Sensor 客制化功能数据类型。

### ➤ 定义

```
typedef enum
{
 E_MI_SNR_CUSTDATA_TO_DRIVER,
 E_MI_SNR_CUSTDATA_TO_USER,
 E_MI_SNR_CUSTDATA_MAX = E_MI_SNR_CUSTDATA_TO_USER,
} MI_SNR_CUST_DIR_e;
```

## ➤ 成员

| 成员名称                        | 描述                           |
|-----------------------------|------------------------------|
| E_MI_SNR_CUSTDATA_TO_DRIVER | 客制化buffer 数据设置给Sensor Driver |
| E_MI_SNR_CUSTDATA_TO_USER   | 客制化buffer 数据从sensor 获取       |
| E_MI_SNR_CUSTDATA_MAX       | 数据类型MAX 选项                   |

## ※ 注意事项

无。

## ➤ 相关数据类型及接口

[MI\\_SNR\\_CustFunction](#)

### 3.12. MI\_SNR\_InitParam\_t

## ➤ 说明

sensor 设备初始化参数。

## ➤ 定义

```
typedef struct MI_SNR_InitParam_s
{
 MI_U32 u32DevId;
 MI_U8 *u8Data;
} MI_SNR_InitParam_t;
```

## ➤ 成员

| 成员名称     | 描述         |
|----------|------------|
| u32DevId | 设备ID       |
| u8Data   | 数据指针buffer |

## ※ 注意事项

无。

## ➤ 相关数据类型及接口

[MI\\_SNR\\_InitDev](#)

### 3.13. MI\_SNR\_Anadec\_SrcAttr\_t

## ➤ 说明

定义模拟信号输入参数。

## ➤ 定义

```
typedef struct MI_SNR_Anadec_SrcAttr_s
{
 MI_SNR_Anadec_Status_e eStatus;
 MI_SNR_Anadec_TransferMode_e eTransferMode;
```

```

MI_SNR_Anadec_Format_e eFormat;
MI_SYS_WindowSize_t stRes;
MI_U32 u32Fps;
}MI_SNR_Anadec_SrcAttr_t;

```

➤ 成员

| 成员名称          | 描述        |
|---------------|-----------|
| eStatus       | 输入源状态枚举   |
| eTransferMode | 输入源传输模式枚举 |
| eFormat       | 输入源传输制式枚举 |
| stRes         | 输入源分辨率    |
| u32Fps        | 输入源帧率     |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_GetAnadecSrcAttr](#)

### 3.14. MI\_SNR\_Anadec\_Status\_e

➤ 说明

定义模拟信号输入源状态。

➤ 定义

```

typedef enum
{
 E_MI_SNR_ANADEC_STATUS_NO_READY = 0,
 E_MI_SNR_ANADEC_STATUS_DISCNT,
 E_MI_SNR_ANADEC_STATUS_CONNECT,
 E_MI_SNR_ANADEC_STATUS_NUM
}MI_SNR_Anadec_Status_e;

```

➤ 成员

| 成员名称                            | 描述         |
|---------------------------------|------------|
| E_MI_SNR_ANADEC_STATUS_NO_READY | 输入源未准备就绪   |
| E_MI_SNR_ANADEC_STATUS_DISCNT   | 输入源状态未连接   |
| E_MI_SNR_ANADEC_STATUS_CONNECT  | 输入源状态已连接   |
| E_MI_SNR_ANADEC_STATUS_NUM      | 输入源状态NUM选项 |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_Anadec\\_SrcAttr\\_t](#)

[MI\\_SNR\\_GetAnadecSrcAttr](#)

### 3.15. MI\_SNR\_Anadec\_TransferMode\_e

➤ 说明

定义模拟信号输入源传输模式。

➤ 定义

```
typedef enum
{
 E_MI_SNR_ANADEC_TRANSFERMODE_CVBS = 0,
 E_MI_SNR_ANADEC_TRANSFERMODE_CVI,
 E_MI_SNR_ANADEC_TRANSFERMODE_TVI,
 E_MI_SNR_ANADEC_TRANSFERMODE_AHD,
 E_MI_SNR_ANADEC_TRANSFERMODE_NUM,
}MI_SNR_Anadec_TransferMode_e;
```

➤ 成员

| 成员名称                              | 描述        |
|-----------------------------------|-----------|
| E_MI_SNR_ANADEC_TRANSFERMODE_CVBS | CVBS传输模式  |
| E_MI_SNR_ANADEC_TRANSFERMODE_CVI  | CVI传输模式   |
| E_MI_SNR_ANADEC_TRANSFERMODE_TVI  | TVI传输模式   |
| E_MI_SNR_ANADEC_TRANSFERMODE_AHD  | AHD传输模式   |
| E_MI_SNR_ANADEC_TRANSFERMODE_NUM  | 传输模式NUM选项 |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_Anadec\\_SrcAttr\\_t](#)

[MI\\_SNR\\_GetAnadecSrcAttr](#)

### 3.16. MI\_SNR\_Anadec\_Format\_e

➤ 说明

定义模拟信号输入源制式。

➤ 定义

```
typedef enum
{
 E_MI_SNR_ANADEC_FORMAT_NTSC = 0,
 E_MI_SNR_ANADEC_FORMAT_PAL,
 E_MI_SNR_ANADEC_FORMAT_NUM,
}MI_SNR_Anadec_Format_e;
```

➤ 成员

| 成员名称                        | 描述        |
|-----------------------------|-----------|
| E_MI_SNR_ANADEC_FORMAT_NTSC | NTSC制式    |
| E_MI_SNR_ANADEC_FORMAT_PAL  | PAL制式     |
| E_MI_SNR_ANADEC_FORMAT_NUM  | 传输制式NUM选项 |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_Anadec\\_SrcAttr\\_t](#)

[MI\\_SNR\\_GetAnadecSrcAttr](#)

### 3.17. [MI\\_SNR\\_Polar\\_e](#)

➤ 说明

定义视频输入信号有效。

➤ 定义

```
typedef enum
{
 E_MI_SNR_PIN_POLAR_POS,
 E_MI_SNR_PIN_POLAR_NEG
} MI_SNR_Polar_e;
```

➤ 成员

| 成员名称                   | 描述      |
|------------------------|---------|
| E_MI_SNR_PIN_POLAR_POS | 高电平信号有效 |
| E_MI_SNR_PIN_POLAR_NEG | 低电平信号有效 |

※ 注意事项

只有 parallel sensor 接口可以设置

➤ 相关数据类型及接口

[MI\\_SNR\\_SyncAttr\\_t](#)

### 3.18. [MI\\_SNR\\_SyncAttr\\_t](#)

➤ 说明

同步信号属性设置。

➤ 定义

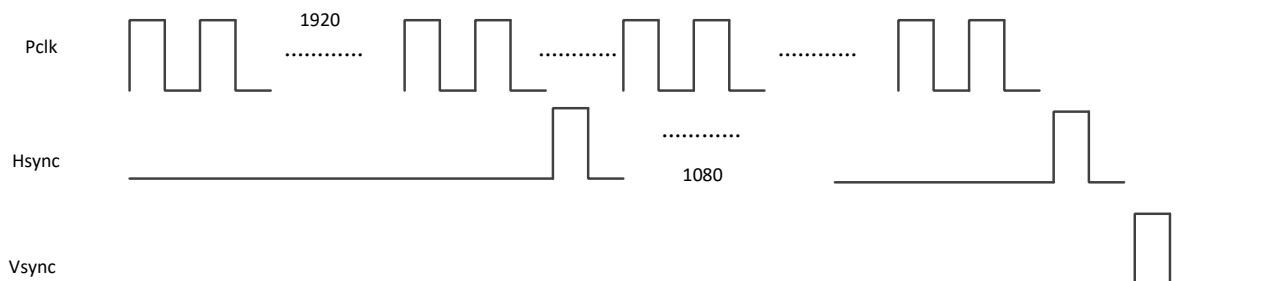
```
typedef struct MI_SNR_SyncAttr_s
{
 MI_SNR_Polar_e eVsyncPolarity;
```

```
MI_SNR_Polar_e eHsyncPolarity;
MI_SNR_Polar_e ePclkPolarity;
MI_U32 VsyncDelay;
MI_U32 HsyncDelay;
MI_U32 PclkDelay;
} MI_SNR_SyncAttr_t;
```

➤ 成员

| 成员名称           | 描述               |
|----------------|------------------|
| eVsyncPolarity | 垂直同步信号有效极性       |
| eHsyncPolarity | 水平同步信号有效极性       |
| ePclkPolarity  | Pixel Clock 有效极性 |
| VsyncDelay     | 垂直同步信号延时时间       |
| HsyncDelay     | 水平同步信号延时时间       |
| PclkDelay      | Pixel Clock 延时时间 |

以 1920x1080 分辨率， Pclk/Hsync/Vsync 都是高电平有效，波形对应关系如下：



Pclk: pixel clock, 每收到一个 pixel, 产生一个 clock。

Hsync:行同步，有效时收到的信号属于同一行。

Vsync:场同步，有效时收到的信号属于同一张 frame。

※ 注意事项

只有 parallel sensor 接口可以设置

➤ 相关数据类型及接口

[MI\\_SNR\\_AttrParallel\\_t](#)

### 3.19. MI\_SNR\_BitOrder\_e

➤ 说明

设备的 data bit 顺序翻转设定。

➤ 定义

typedef enum

```
{
 E_MI_SNR_BITORDER_NORMAL = 0,
 E_MI_SNR_BITORDER_REVERSED
} MI_SNR_BitOrder_e;
```

➤ 成员

| 成员名称                       | 描述           |
|----------------------------|--------------|
| E_MI_SNR_BITORDER_NORMAL   | 正常data bit排序 |
| E_MI_SNR_BITORDER_REVERSED | 逆序data bit排序 |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_AttrBt656\\_t](#)

### 3.20. MI\_SNR\_ClkEdge\_e

➤ 说明

定义视频设备接收的时钟类型。

➤ 定义

```
typedef enum
{
 E_MI_SNR_CLK_EDGE_SINGLE_UP,
 E_MI_SNR_CLK_EDGE_SINGLE_DOWN,
 E_MI_SNR_CLK_EDGE_DOUBLE,
 E_MI_SNR_CLK_EDGE_MAX
} MI_SNR_ClkEdge_e;
```

➤ 成员

| 成员名称                          | 描述                     |
|-------------------------------|------------------------|
| E_MI_SNR_CLK_EDGE_SINGLE_UP   | 时钟单沿模式，且 VIF 设备在上升沿采样。 |
| E_MI_SNR_CLK_EDGE_SINGLE_DOWN | 时钟单沿模式，且 VIF 设备在下降沿采样。 |
| E_MI_SNR_CLK_EDGE_DOUBLE      | 前端送过来双沿数据时，VIF 进行双沿采样。 |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_SNR\\_AttrBt656\\_t](#)

### 3.21. MI\_SNR\_IntfMode\_e

➤ 说明



定义视频输入设备的接口模式

➤ 定义

```
typedef enum
{
 E_MI_SNR_MODE_BT656,
 E_MI_SNR_MODE_DIGITAL_CAMERA,
 E_MI_SNR_MODE_BT1120_STANDARD,
 E_MI_SNR_MODE_BT1120_INTERLEAVED,
 E_MI_SNR_MODE_MIPI,
 E_MI_SNR_MODE_LVDS,
 E_MI_SNR_MODE_MAX
} MI_SNR_IntfMode_e;
```

➤ 成员

| 成员名称                             | 描述                                                                             |
|----------------------------------|--------------------------------------------------------------------------------|
| E_MI_SNR_MODE_BT656              | 输入数据的协议符合标准 <b>BT.656</b> 协议，端口数据输入模式为亮度色度复合模式，分量模式为单分量。                       |
| E_MI_SNR_MODE_DIGITAL_CAMERA     | 输入数据的协议为 <b>Digital camera</b> 协议，端口数据输入模式为亮度色度复合模式，分量模式为单分量。                  |
| E_MI_SNR_MODE_BT1120_STANDARD    | 输入数据的协议符合标准 <b>BT.1120</b> 协议（ <b>BT.656+</b> 双分量），端口数据输入模式为亮度色度分离模式，分量模式为双分量。 |
| E_MI_SNR_MODE_BT1120_INTERLEAVED | 输入数据的协议符合 <b>BT.1120 interleave</b> 模式，端口数据输入模式为亮度色度分离模式，分量模式为双分量。             |
| E_MI_SNR_MODE_MIPI               | 输入数据符合 <b>MIPI</b> 协议                                                          |
| E_MI_SNR_MODE_LVDS               | 输入数据符合 <b>LVDS</b> 协议                                                          |

※ 注意事项

可以通过 **MI\_SNR\_GetPadInfo** 中的 **eIntfMode** 获取当前接口模式。

➤ 相关数据类型及接口

[MI\\_SNR\\_PADInfo\\_t](#)

## 3.22. MI\_SNR\_HDRType\_e

➤ 说明

定义视频设备 **HDR** 类型。

➤ 定义

```
typedef enum
```

```
{
 E_MI_SNR_HDR_TYPE_OFF,
 E_MI_SNR_HDR_TYPE_VC, //virtual channel mode HDR,vc0->long, vc1->short
 E_MI_SNR_HDR_TYPE_DOL,
 E_MI_SNR_HDR_TYPE_EMBEDDED, //compressed HDR mode
 E_MI_SNR_HDR_TYPE_LI, //Line interlace HDR
 E_MI_SNR_HDR_TYPE_MAX
} MI_SNR_HDRType_e;
```

➤ 成员

| 成员名称                       | 描述                                             |
|----------------------------|------------------------------------------------|
| E_MI_SNR_HDR_TYPE_OFF      | 不开HDR                                          |
| E_MI_SNR_HDR_TYPE_VC       | virtual channel mode HDR,vc0->long, vc1->short |
| E_MI_SNR_HDR_TYPE_DOL      | Digital Overlap High Dynamic Range             |
| E_MI_SNR_HDR_TYPE_EMBEDDED | compressed HDR mode                            |
| E_MI_SNR_HDR_TYPE_LI       | Line interlace HDR                             |

※ 注意事项

hdr type 和 sensor 相关， 可以通过 MI\_SNR\_GetPadInfo 的 eHDRMode 获取当前 sensor 支持的 HDR type。

➤ 相关数据类型及接口

[MI\\_SNR\\_PADInfo\\_t](#)

## 4. SENSOR 错误码

视频输入 API 错误码如下表所示。

| 错误代码       | 宏定义                         | 描述         |
|------------|-----------------------------|------------|
| 0xA01B2001 | MI_ERR_SNR_INVALID_DEVID    | 设备号无效      |
| 0xA01B2002 | MI_ERR_SNR_INVALID_CHNID    | 通道号无效      |
| 0xA01B2003 | MI_ERR_SNR_INVALID_PARA     | 参数设置无效     |
| 0xA01B2006 | MI_ERR_SNR_INVALID_NULL_PTR | 输入参数空指针错误  |
| 0xA01B2007 | MI_ERR_SNR_FAILED_NOTCONFIG | 设备或通道属性未配置 |
| 0xA01B2008 | MI_ERR_SNR_NOT_SUPPORT      | 操作不支持      |
| 0xA01B2009 | MI_ERR_SNR_NOT_PERM         | 操作不允许      |
| 0xA01B200C | MI_ERR_SNR_NOMEM            | 分配内存失败     |
| 0xA01B200E | MI_ERR_SNR_BUF_EMPTY        | Buffer空    |
| 0xA01B200F | MI_ERR_SNR_BUF_FULL         | Buffer满    |
| 0xA01B2010 | MI_ERR_SNR_SYS_NOTREADY     | 系统未初始化     |
| 0xA01B2012 | MI_ERR_SNR_BUSY             | 系统忙        |
| 0xA01B201F | MI_ERR_SNR_FAIL             | 端口无效       |

## 5. PROCFS 介绍

### 5.1. cat

- 调试信息

```
cat /proc/mi_modules/mi_sensor/mi_sensor0
```

```
----- start dump Pad info -----
PadId PlaneMode bEnable bmirror bflip fps ResCnt intfmode hdrmode planeCnt
0 0 1 0 0 30 1 MIPI 2 2
MipiAttr YuvOrder HdrHwMode DataFmt HdrVchNum HsyncMode LaneNum LPackType0 LPackType1 samDelay
0 0 1 0 0 0 0 0 0 0
Res strResDesc CropX CropY CropW CropH OutW OutH MaxFps MinFps
1920x1080@30fps 0 0 1920 1080 1948 1097 30 3
Cur 1920x1080@30fps 0 0 1920 1080 1948 1097 30 3
----- End dump Pad info -----

----- start dump plane info -----
PadId PlaneId SnrName BayerId ePixPrec eHdrSrc CropX CropY CropW CropH
0 0 IMX307_MIPI RG 12BPP 0 0 0 1920 1080
0 0 IMX307_MIPI RG 12BPP 0 0 0 1920 1080
```

- 调试信息分析

记录当前 sensor 的使用状况以及相关属性、可以动态地获取到这些信息，方便调试和测试。

- 参数说明

| 参数       |           | 描述              |                       |
|----------|-----------|-----------------|-----------------------|
| Pad info | Padid     | Sensor pad id   |                       |
|          | Planemode | Planemode 开关    |                       |
|          | bEnable   | Sensor enable   |                       |
|          | bmirror   | 水平翻转            |                       |
|          | bflip     | 垂直翻转            |                       |
|          | fps       | 帧率              |                       |
|          | ResCnt    | Sensor 支持分辨率的数量 |                       |
|          | Intfmode  | 接口协议            |                       |
|          | Hdrmode   | Hdr hw mode     |                       |
|          | PlaneCnt  | Plane 通道数量      |                       |
|          | Mipiattr  | Yuvorder        | Yuv 数据排列              |
|          |           | Hdrhwmode       | Hdr hw mode           |
|          |           | Datafmt         | 数据格式，yuv/rgb          |
|          |           | HdrVchNum       | Sensor 支持的 HDR 虚拟通道数量 |

| 参数 |              | 描述                          |                                    |
|----|--------------|-----------------------------|------------------------------------|
|    |              | HsyncMode                   | 0:同步前一条<br>1:同步后一条 line 的 hsync 信号 |
|    |              | LaneNum                     | 支持同时传输数据的信号线数量                     |
|    |              | LPackType0                  | 数据打包格式 0                           |
|    |              | LPackType1                  | 数据打包格式 1                           |
|    |              | SamDelay                    | 延时跳过数据头部分                          |
|    | BT656Attr    | BitSwap                     | 数据排列方向                             |
|    |              | ClkEdge                     | 采样时钟模式                             |
|    |              | Multinum                    | 复合模式的路数                            |
|    |              | HPol                        | H sync 极性                          |
|    |              | PcPol                       | Pclk 极性                            |
|    |              | VPol                        | V sync 极性                          |
|    |              | HDelay                      | H sync delay                       |
|    |              | VDelay                      | V sync delay                       |
|    |              | PcDelay                     | Pclk delay                         |
|    | ParallelAttr | HPol                        | H sync 极性                          |
|    |              | PcPol                       | Pclk 极性                            |
|    |              | VPol                        | V sync 极性                          |
|    |              | HDelay                      | H sync delay                       |
|    |              | VDelay                      | V sync delay                       |
|    |              | PcDelay                     | Pclk delay                         |
|    | Res          | Sensor 支持的分辨率信息             |                                    |
|    |              | strResDesc                  | 分辨率                                |
|    |              | CropX/CropY/CropW/<br>CropH | Crop 参数                            |
|    |              | OutW/OutH                   | 输出的宽高                              |
|    |              | MaxFps/MinFps               | 最大最小帧率                             |
|    | Cur          | 当前使用的分辨率信息                  |                                    |
|    | Padid        | pad id                      |                                    |
|    | Planeid      | Plane id                    |                                    |
|    | SnrName      | Sensor name                 |                                    |

| 参数       |                             | 描述                           |
|----------|-----------------------------|------------------------------|
| Pad info | BayerId                     | Bayer id                     |
|          | ePixPrec                    | Pixel 精确度 (12bpp,10bpp ....) |
|          | eHdrSrc                     | HDR 通道号                      |
|          | CropX/CropY/CropW/<br>CropH | Crop 参数                      |

## 5.2. echo

# echo help > /proc/mi\_modules/mi\_sensor/mi\_sensor0

```
-----MI_SYS_IMPL_Common_Help end-----

setmirrorflip [PadId, bMirror, bFlip];
setfps [Padid, fps];
```

Echo help 查看可用命令

|      |                                                                                    |
|------|------------------------------------------------------------------------------------|
| 功能   | 设置 sensor 的翻转                                                                      |
| 命令   | echo setmirrorflip [PadId, bMirror, bFlip] > /proc/mi_modules/mi_sensor/mi_sensor0 |
| 参数说明 | PadId: pad id                                                                      |
|      | bMirror: 水平翻转                                                                      |
|      | bFlip: 上下翻转                                                                        |
| 举例   | echo setmirrorflip 0 1 0 > /proc/mi_modules/mi_sensor/mi_sensor0                   |

|      |                                                                  |
|------|------------------------------------------------------------------|
| 功能   | 设置 sensor 的帧率                                                    |
| 命令   | echo setfps [Padid, fps] > /proc/mi_modules/mi_sensor/mi_sensor0 |
| 参数说明 | PadId: pad id                                                    |
|      | fps: 帧率                                                          |
| 举例   | echo setfps 0 15 > /proc/mi_modules/mi_sensor/mi_sensor0         |